

CHOCOLATE SALES ANALYSIS

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

choco = pd.read_csv(r"C:\Users\chukw\Documents\RAIN\AIML 1ST SEMESTER
(File responses)\exam aiml 111\Chocolate Sales (2).csv")
```

```
choco.head()
```

	Sales Person	Country	Product	Date
Amount \				
0	Jehu Rudeforth	UK	Mint Chip Choco	04/01/2022
\$5,320.00				
1	Van Tuxwell	India	85% Dark Bars	01/08/2022
\$7,896.00				
2	Gigi Bohling	India	Peanut Butter Cubes	07/07/2022
\$4,501.00				
3	Jan Morforth	Australia	Peanut Butter Cubes	27/04/2022
\$12,726.00				
4	Jehu Rudeforth	UK	Peanut Butter Cubes	24/02/2022
\$13,685.00				

	Boxes Shipped
0	180
1	94
2	91
3	342
4	184

DATA CLEANING AND PROCESSING

```
choco.dtypes
```

Sales Person	object
Country	object
Product	object
Date	object
Amount	object
Boxes Shipped	int64
dtype:	object

```
choco.isnull().sum()
```

Sales Person	0
Country	0
Product	0
Date	0

```
Amount          0
Boxes Shipped    0
dtype: int64
```

data featuring

```
"$12,726.00".replace("$", "").replace(",", "")
```

```
'12726.00'
```

```
def money(value):
    try:
        return value.replace("$", "").replace(",", "")
    except:
        return value
```

```
choco["Amount"] = choco["Amount"].apply(money)
```

```
choco
```

	Sales Person	Country	Product	Date
Amount \				
0	Jehu Rudeforth	UK	Mint Chip Choco	04/01/2022
5320.00				
1	Van Tuxwell	India	85% Dark Bars	01/08/2022
7896.00				
2	Gigi Bohling	India	Peanut Butter Cubes	07/07/2022
4501.00				
3	Jan Morforth	Australia	Peanut Butter Cubes	27/04/2022
12726.00				
4	Jehu Rudeforth	UK	Peanut Butter Cubes	24/02/2022
13685.00				
...	...	...	...	...
...				
3277	Karlen McCaffrey	Australia	Spicy Special Slims	17/05/2024
5303.58				
3278	Jehu Rudeforth	USA	White Choc	07/06/2024
7339.32				
3279	Ches Bonnell	Canada	Organic Choco Syrup	26/07/2024
616.09				
3280	Dotty Strutley	India	Eclairs	28/07/2024
2504.62				
3281	Karlen McCaffrey	India	70% Dark Bites	23/05/2024
5915.87				

	Boxes Shipped
0	180
1	94
2	91
3	342

```

4          184
...
3277       354
3278       121
3279       238
3280       397
3281       355

```

```
[3282 rows x 6 columns]
```

```

coltochange=["Amount","Boxes Shipped"]
for col in choco.columns:
    if col in coltochange:
        choco[col] = pd.to_numeric(choco[col], errors = "coerce")

choco["Date"]=
pd.to_datetime(choco["Date"],errors="coerce",dayfirst=True)

```

```
choco
```

	Sales Person	Country	Product	Date
Amount \				
0	Jehu Rudeforth	UK	Mint Chip Choco	2022-01-04
5320.00				
1	Van Tuxwell	India	85% Dark Bars	2022-08-01
7896.00				
2	Gigi Bohling	India	Peanut Butter Cubes	2022-07-07
4501.00				
3	Jan Morforth	Australia	Peanut Butter Cubes	2022-04-27
12726.00				
4	Jehu Rudeforth	UK	Peanut Butter Cubes	2022-02-24
13685.00				
...	...	...	...	...
...				
3277	Karlen McCaffrey	Australia	Spicy Special Slims	2024-05-17
5303.58				
3278	Jehu Rudeforth	USA	White Choc	2024-06-07
7339.32				
3279	Ches Bonnell	Canada	Organic Choco Syrup	2024-07-26
616.09				
3280	Dotty Strutley	India	Eclairs	2024-07-28
2504.62				
3281	Karlen McCaffrey	India	70% Dark Bites	2024-05-23
5915.87				

	Boxes Shipped
0	180
1	94
2	91
3	342

```

4          184
...
3277       354
3278       121
3279       238
3280       397
3281       355

```

```
[3282 rows x 6 columns]
```

```
choco.dtypes
```

```

Sales Person      object
Country           object
Product           object
Date              datetime64[ns]
Amount            float64
Boxes Shipped     int64
dtype: object

```

```

choco["Day"] = choco["Date"].dt.day
choco["Month"] = choco["Date"].dt.month
choco["year"] = choco["Date"].dt.year
choco["Total"] = choco["Amount"] * choco["Boxes Shipped"]

```

```
choco.head()
```

	Sales Person	Country	Product	Date	Amount
0	Jehu Rudeforth	UK	Mint Chip Choco	2022-01-04	5320.0
1	Van Tuxwell	India	85% Dark Bars	2022-08-01	7896.0
2	Gigi Bohling	India	Peanut Butter Cubes	2022-07-07	4501.0
3	Jan Morforth	Australia	Peanut Butter Cubes	2022-04-27	12726.0
4	Jehu Rudeforth	UK	Peanut Butter Cubes	2022-02-24	13685.0

	Boxes Shipped	Day	Month	year	Total
0	180	4	1	2022	957600.0
1	94	1	8	2022	742224.0
2	91	7	7	2022	409591.0
3	342	27	4	2022	4352292.0
4	184	24	2	2022	2518040.0

EXPLORATORY DATA ANALYSIS/VISUALISATION

Q1 Product with the highest sales over the period of July 2022-december 2025

```
period = (choco["Date"] >= "2022-07-01") & (choco["Date"] <= "2024-12-31")
```

```
productdf = choco.loc[period].groupby("Product")[["Total"]].sum()
```

```
highestproduct= productdf["Total"].max()
```

```
highestproduct
```

```
135027786.17
```

```
highestSales = productdf.loc[productdf["Total"]==highestproduct]
```

```
highestSales
```

	Total
Product	
Smooth Sliky Salty	1.350278e+08

```
sortprod = productdf.sort_values(by="Total", ascending= False)
```

```
top5=sortprod.head(5)
```

```
top5
```

	Total
Product	
Smooth Sliky Salty	1.350278e+08
50% Dark Bites	1.330923e+08
Peanut Butter Cubes	1.302638e+08
99% Dark & Pure	1.263781e+08
Mint Chip Choco	1.258466e+08

```
plt.bar(top5.index,top5["Total"])
```

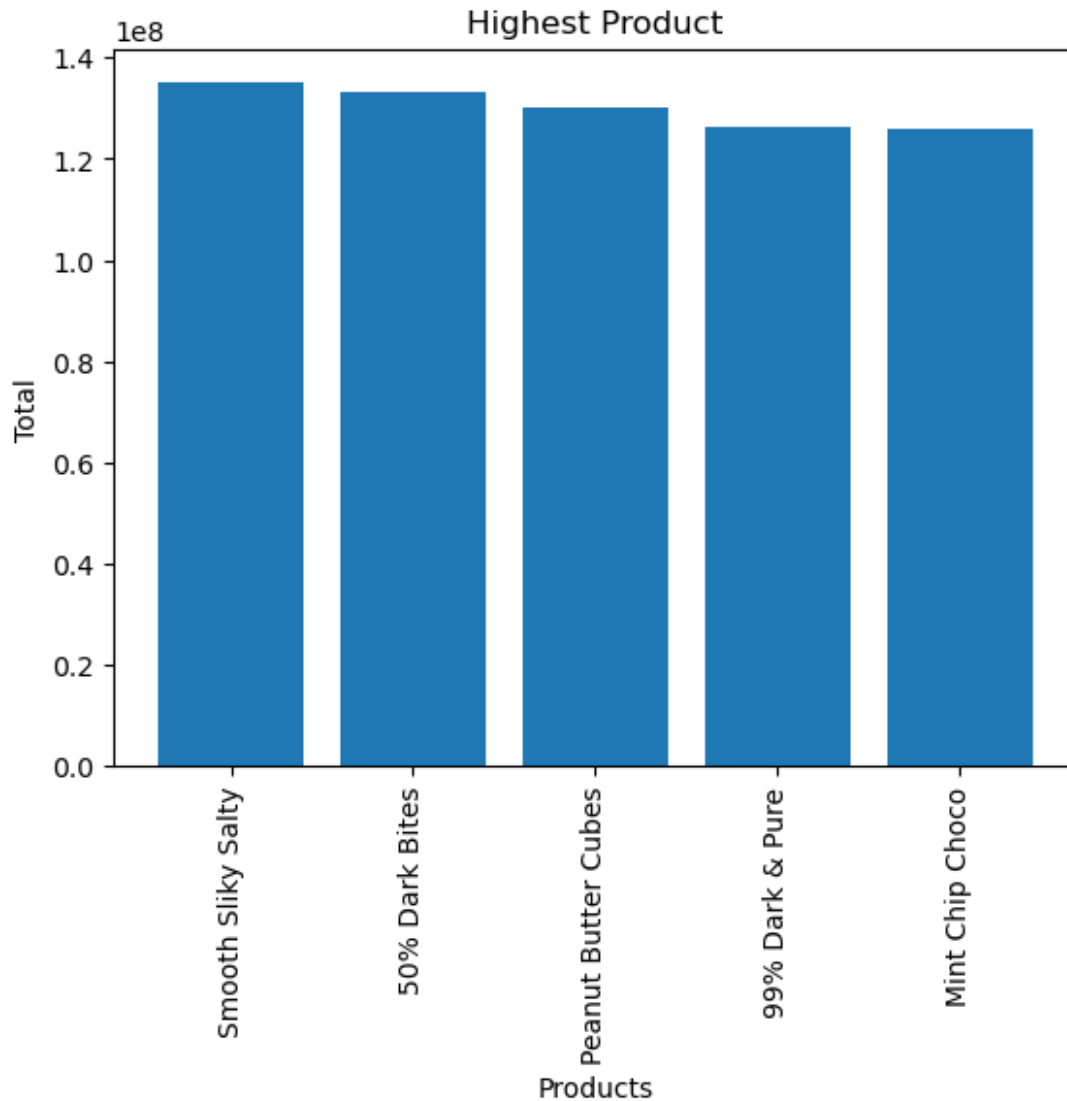
```
plt.xticks(rotation="vertical")
```

```
plt.title("Highest Product")
```

```
plt.xlabel("Products")
```

```
plt.ylabel("Total")
```

```
plt.show()
```



From the period of July2022-December2025, the product smooth silky salty was the most sold product

Q2 COUNTRY LEVEL-SALES COMPARISON

```
choco.head()
```

	Sales Person	Country	Product	Date	Amount
0	Jehu Rudeforth	UK	Mint Chip Choco	2022-01-04	5320.0
1	Van Tuxwell	India	85% Dark Bars	2022-08-01	7896.0
2	Gigi Bohling	India	Peanut Butter Cubes	2022-07-07	4501.0
3	Jan Morforth	Australia	Peanut Butter Cubes	2022-04-27	12726.0

```
4 Jehu Rudeforth          UK Peanut Butter Cubes 2022-02-24 13685.0
```

	Boxes Shipped	Day	Month	year	Total
0	180	4	1	2022	957600.0
1	94	1	8	2022	742224.0
2	91	7	7	2022	409591.0
3	342	27	4	2022	4352292.0
4	184	24	2	2022	2518040.0

```
countrydf = choco.groupby("Country")["Total"].sum()  
countrydf
```

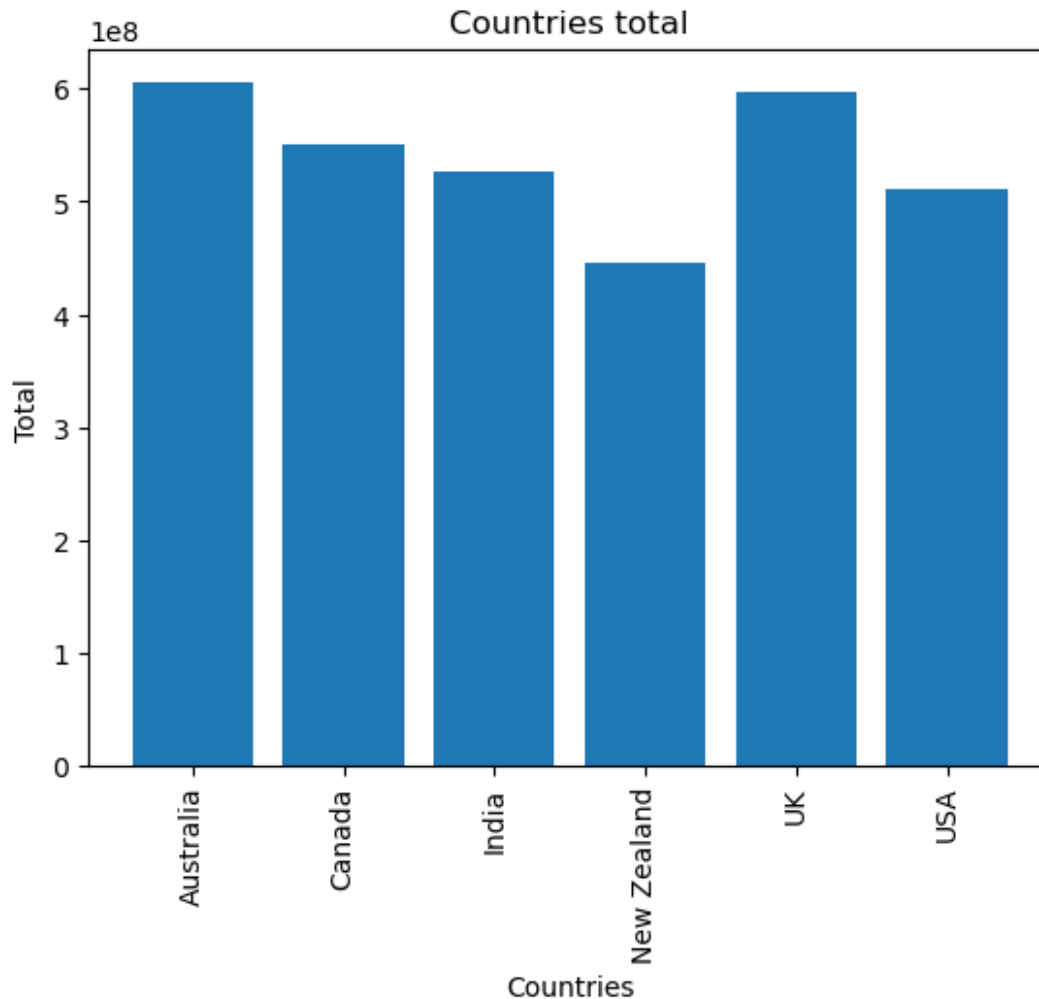
	Total
Country	
Australia	6.051231e+08
Canada	5.510098e+08
India	5.260366e+08
New Zealand	4.456097e+08
UK	5.967866e+08
USA	5.109330e+08

```
countrydf.index
```

```
Index(['Australia', 'Canada', 'India', 'New Zealand', 'UK', 'USA'],  
      dtype='object', name='Country')
```

Highest and Lowest Performing Countries in bar chart

```
plt.bar(countrydf.index, countrydf["Total"])  
plt.xticks(rotation="vertical")  
plt.title("Countries total")  
plt.xlabel("Countries")  
plt.ylabel("Total")  
plt.show()
```



From the data:

highest performing country = Australia

lowest performing country = New Zealand

Percentage contribution to total sales

```
countrydf = choco.groupby("Country")[["Total"]].sum()
```

```
countrydf["Percentage"] = (countrydf["Total"] /  
countrydf["Total"].sum()) * 100
```

```
plt.pie?
```

Signature:

```
plt.pie(  
    x: 'ArrayLike',
```

```

*,
explode: 'ArrayLike | None' = None,
labels: 'Sequence[str] | None' = None,
colors: 'ColorType | Sequence[ColorType] | None' = None,
autopct: 'str | Callable[[float], str] | None' = None,
pctdistance: 'float' = 0.6,
shadow: 'bool' = False,
labeldistance: 'float | None' = 1.1,
startangle: 'float' = 0,
radius: 'float' = 1,
counterclock: 'bool' = True,
wedgeprops: 'dict[str, Any] | None' = None,
textprops: 'dict[str, Any] | None' = None,
center: 'tuple[float, float]' = (0, 0),
frame: 'bool' = False,
rotatelabels: 'bool' = False,
normalize: 'bool' = True,
hatch: 'str | Sequence[str] | None' = None,
data=None,
) -> 'tuple[list[Wedge], list[Text]] | tuple[list[Wedge], list[Text],
list[Text]]'
Docstring:
Plot a pie chart.

```

Make a pie chart of array `*x*`. The fractional area of each wedge is given by ```x/sum(x)```.

The wedges are plotted counterclockwise, by default starting from the x-axis.

Parameters

`x` : 1D array-like
The wedge sizes.

`explode` : array-like, default: None
If not `*None*`, is a ```len(x)``` array which specifies the fraction of the radius with which to offset each wedge.

`labels` : list, default: None
A sequence of strings providing the labels for each wedge

`colors` : :mpltype:`color` or list of :mpltype:`color`, default: None
A sequence of colors through which the pie chart will cycle. If `*None*`, will use the colors in the currently active cycle.

`hatch` : str or list, default: None
Hatching pattern applied to all pie wedges or sequence of patterns through which the chart will cycle. For a list of valid patterns, see :doc:`/gallery/shapes\_and\_collections/hatch\_style\_reference`.

.. versionadded:: 3.7

`autopct` : None or str or callable, default: None

If not `*None*`, `*autopct*` is a string or function used to label the wedges with their numeric value. The label will be placed inside the wedge. If `*autopct*` is a format string, the label will be ```fmt % pct```. If `*autopct*` is a function, then it will be called.

`pctdistance` : float, default: 0.6

The relative distance along the radius at which the text generated by `*autopct*` is drawn. To draw the text outside the pie, set `*pctdistance*` > 1. This parameter is ignored if `*autopct*` is ```None```.

`labeldistance` : float or None, default: 1.1

The relative distance along the radius at which the labels are drawn. To draw the labels inside the pie, set `*labeldistance*` < 1.

If set to ```None```, labels are not drawn but are still stored for use in ``.legend``.

`shadow` : bool or dict, default: False

If bool, whether to draw a shadow beneath the pie. If dict, draw a shadow

passing the properties in the dict to ``.Shadow``.

.. versionadded:: 3.8

`*shadow*` can be a dict.

`startangle` : float, default: 0 degrees

The angle by which the start of the pie is rotated, counterclockwise from the x-axis.

`radius` : float, default: 1

The radius of the pie.

`counterclock` : bool, default: True

Specify fractions direction, clockwise or counterclockwise.

`wedgeprops` : dict, default: None

Dict of arguments passed to each ``.patches.Wedge`` of the pie. For example, ```wedgeprops = {'linewidth': 3}``` sets the width of the wedge border lines equal to 3. By default, ``clip_on=False``. When there is a conflict between these properties and other keywords, properties passed to *wedgeprops* take precedence.`

`textprops` : dict, default: None

Dict of arguments to pass to the text objects.

center : (float, float), default: (0, 0)
 The coordinates of the center of the chart.

frame : bool, default: False
 Plot Axes frame with the chart if true.

rotatelabels : bool, default: False
 Rotate each label to the angle of the corresponding slice if true.

normalize : bool, default: True
 When *True*, always make a full pie by normalizing x so that
 ``sum(x) == 1``. *False* makes a partial pie if ``sum(x) <= 1``
 and raises a `ValueError` for ``sum(x) > 1``.

data : indexable object, optional
 If given, the following parameters also accept a string ``s``,
 which is
 interpreted as ``data[s]`` if ``s`` is a key in ``data``:

x, *explode*, *labels*, *colors*

Returns

patches : list
 A sequence of `matplotlib.patches.Wedge` instances

texts : list
 A list of the label `.Text` instances.

autotexts : list
 A list of `.Text` instances for the numeric labels. This will only
 be returned if the parameter *autopct* is not *None*.

Notes

.. note::

This is the :ref:`pyplot wrapper <pyplot\_interface>` for
 `.axes.Axes.pie`.

The pie chart will probably look best if the figure and Axes are
 square, or the Axes aspect is equal.

This method sets the aspect ratio of the axis to "equal".

The Axes aspect ratio can be controlled with `.Axes.set\_aspect`.

File: c:\users\chukw\anaconda3\lib\site-packages\matplotlib\pyplot.py

Type: function

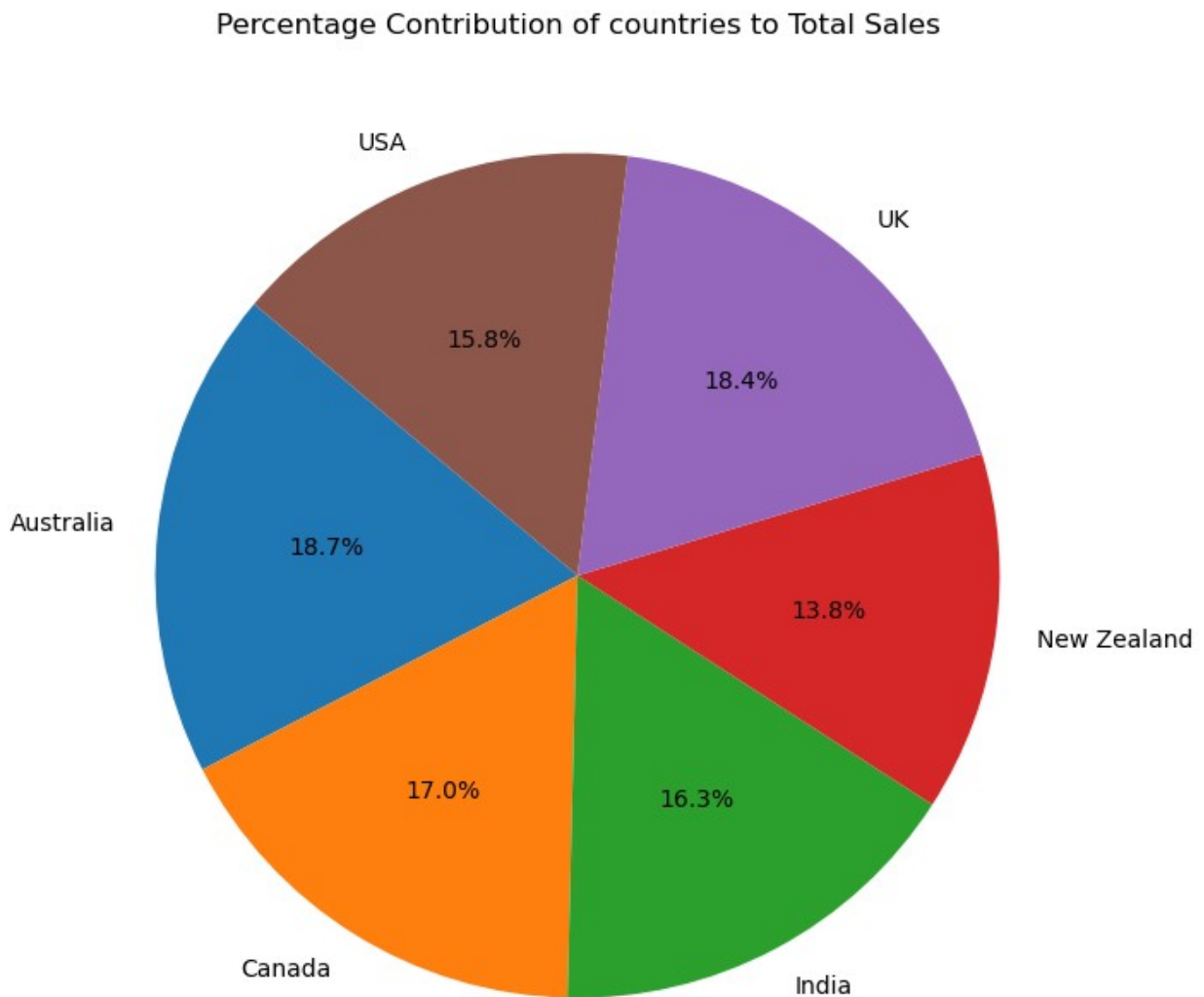
```
plt.figure(figsize=(8,8))
```

```
plt.pie(
```

```

countrydf["Total"],
labels=countrydf.index,
autopct="%1.1f%%", # shows percentage on chart
startangle=140
)
plt.title("Percentage Contribution of countries to Total Sales")
plt.show()

```



from the data, the following countries can be ranked in the following order

1. AUstralia
2. UK
3. USA
4. Canada
5. India

6. New zealand

Q3 Time Series and Trend analysis

```
monthdf = choco.groupby(["year", "Month"])[["Total"]].sum()  
monthdf
```

		Total
year	Month	
2022	1	1.555305e+08
	2	1.121084e+08
	3	1.130568e+08
	4	1.198464e+08
	5	1.242704e+08
	6	1.377418e+08
	7	1.194995e+08
	8	1.081836e+08
2023	1	1.713028e+08
	2	1.221479e+08
	3	1.252424e+08
	4	1.351019e+08
	5	1.377376e+08
	6	1.538941e+08
	7	1.319026e+08
	8	1.174890e+08
2024	1	1.819212e+08
	2	1.295499e+08
	3	1.314668e+08
	4	1.385795e+08
	5	1.443928e+08
	6	1.613600e+08
	7	1.386053e+08
	8	1.245674e+08

```
monthdf.index
```

```
MultiIndex([(2022, 1),  
            (2022, 2),  
            (2022, 3),  
            (2022, 4),  
            (2022, 5),  
            (2022, 6),  
            (2022, 7),  
            (2022, 8),  
            (2023, 1),  
            (2023, 2),  
            (2023, 3),  
            (2023, 4),  
            (2023, 5),  
            (2023, 6),
```

```

        (2023, 7),
        (2023, 8),
        (2024, 1),
        (2024, 2),
        (2024, 3),
        (2024, 4),
        (2024, 5),
        (2024, 6),
        (2024, 7),
        (2024, 8)],
        names=['year', 'Month'])

MMonths = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug',
'Sep', 'Oct', 'Nov', 'Dec']

newindex = []
for yrnumber, monthnumber in monthdf.index:
    month = MMonths[int(monthnumber-1)]
    newindex.append(f"{yrnumber}{month}")

monthdf.index=newindex

monthdf.index

Index(['2022Jan', '2022Feb', '2022Mar', '2022Apr', '2022May',
'2022Jun',
      '2022Jul', '2022Aug', '2023Jan', '2023Feb', '2023Mar',
'2023Apr',
      '2023May', '2023Jun', '2023Jul', '2023Aug', '2024Jan',
'2024Feb',
      '2024Mar', '2024Apr', '2024May', '2024Jun', '2024Jul',
'2024Aug'],
      dtype='object')

plt.plot?

Signature:
plt.plot(
    *args: 'float | ArrayLike | str',
    scalex: 'bool' = True,
    scaley: 'bool' = True,
    data=None,
    **kwargs,
) -> 'list[Line2D]'
Docstring:
Plot y versus x as lines and/or markers.

Call signatures::

    plot([x], y, [fmt], *, data=None, **kwargs)
    plot([x], y, [fmt], [x2], y2, [fmt2], ..., **kwargs)

```

The coordinates of the points or line nodes are given by **x**, **y**.

The optional parameter **fmt** is a convenient way for defining basic formatting like color, marker and linestyle. It's a shortcut string notation described in the **Notes** section below.

```
>>> plot(x, y)          # plot x and y using default line style and
color
>>> plot(x, y, 'bo')    # plot x and y using blue circle markers
>>> plot(y)             # plot y using x as index array 0..N-1
>>> plot(y, 'r+')       # ditto, but with red plusses
```

You can use ``.Line2D`` properties as keyword arguments for more control on the appearance. Line properties and **fmt** can be mixed. The following two calls yield identical results:

```
>>> plot(x, y, 'go--', linewidth=2, markersize=12)
>>> plot(x, y, color='green', marker='o', linestyle='dashed',
...      linewidth=2, markersize=12)
```

When conflicting with **fmt**, keyword arguments take precedence.

****Plotting labelled data****

There's a convenient way for plotting objects with labelled data (i.e. data that can be accessed by index `obj['y']`). Instead of giving the data in **x** and **y**, you can provide the object in the **data** parameter and just give the labels for **x** and **y**:

```
>>> plot('xlabel', 'ylabel', data=obj)
```

All indexable objects are supported. This could e.g. be a ``dict``, a ``pandas.DataFrame`` or a structured numpy array.

****Plotting multiple sets of data****

There are various ways to plot multiple sets of data.

- The most straight forward way is just to call ``plot`` multiple times. Example:

```
>>> plot(x1, y1, 'bo')
>>> plot(x2, y2, 'go')
```

- If **x** and/or **y** are 2D arrays, a separate data set will be drawn for every column. If both **x** and **y** are 2D, they must have the same shape. If only one of them is 2D with shape (N, m) the other must have length N and will be used for every data set m.

Example:

```
>>> x = [1, 2, 3]
>>> y = np.array([[1, 2], [3, 4], [5, 6]])
>>> plot(x, y)
```

is equivalent to:

```
>>> for col in range(y.shape[1]):
...     plot(x, y[:, col])
```

- The third way is to specify multiple sets of `*[x]*`, `*y*`, `*[fmt]*` groups::

```
>>> plot(x1, y1, 'g^', x2, y2, 'g-')
```

In this case, any additional keyword argument applies to all datasets. Also, this syntax cannot be combined with the `*data*` parameter.

By default, each line is assigned a different style specified by a 'style cycle'. The `*fmt*` and line property parameters are only necessary if you want explicit deviations from these defaults. Alternatively, you can also change the style cycle using `:rc:`axes.prop_cycle``.

Parameters

`x, y` : array-like or scalar

The horizontal / vertical coordinates of the data points.
`*x*` values are optional and default to ``range(len(y))``.

Commonly, these parameters are 1D arrays.

They can also be scalars, or two-dimensional (in that case, the columns represent separate data sets).

These arguments cannot be passed as keywords.

`fmt` : str, optional

A format string, e.g. 'ro' for red circles. See the `*Notes*` section for a full description of the format strings.

Format strings are just an abbreviation for quickly setting basic line properties. All of these and more can also be controlled by keyword arguments.

This argument cannot be passed as keyword.

`data` : indexable object, optional
An object with labelled data. If given, provide the label names to plot in `*x*` and `*y*`.

.. note::

Technically there's a slight ambiguity in calls where the second label is a valid `*fmt*`. `plot('n', 'o', data=obj)` could be `plt(x, y)` or `plt(y, fmt)`. In such cases, the former interpretation is chosen, but a warning is issued. You may suppress the warning by adding an empty format string `plot('n', 'o', '', data=obj)`.

Returns

list of `~.Line2D`

A list of lines representing the plotted data.

Other Parameters

`scalex, scaley` : bool, default: True

These parameters determine if the view limits are adapted to the data limits. The values are passed on to `~.axes.Axes.autoscale_view`.

****kwargs** : `~matplotlib.lines.Line2D` properties, optional

`*kwargs` are used to specify properties like a line label (for auto legends), linewidth, antialiasing, marker face color.

Example::

```
>>> plot([1, 2, 3], [1, 2, 3], 'go-', label='line 1', linewidth=2)
>>> plot([1, 2, 3], [1, 4, 9], 'rs', label='line 2')
```

If you specify multiple lines with one plot call, the kwargs apply to all those lines. In case the label object is iterable, each element is used as labels for each set of data.

Here is a list of available `~.Line2D` properties:

Properties:

`agg_filter`: a filter function, which takes a (m, n, 3) float array and a dpi value, and returns a (m, n, 3) array and two offsets from the bottom left corner of the image

`alpha`: scalar or None

`animated`: bool

`antialiased` or `aa`: bool

`clip_box`: `~matplotlib.transforms.BboxBase` or None

`clip_on`: bool

`clip_path`: Patch or (Path, Transform) or None

`color` or `c`: `:mpltype:color`

`dash_capstyle`: `~.CapStyle` or {'butt', 'projecting', 'round'}

```

dash_joinstyle: `JoinStyle` or {'miter', 'round', 'bevel'}
dashes: sequence of floats (on/off ink in points) or (None, None)
data: (2, N) array or two 1D arrays
drawstyle or ds: {'default', 'steps', 'steps-pre', 'steps-mid',
'steps-post'}, default: 'default'
figure: `~matplotlib.figure.Figure` or
`~matplotlib.figure.SubFigure`
fillstyle: {'full', 'left', 'right', 'bottom', 'top', 'none'}
gapcolor: :mpltype:`color` or None
gid: str
in_layout: bool
label: object
linestyle or ls: {'-', '--', '-.', ':', ''}, (offset, on-off-
seq), ...}
linewidth or lw: float
marker: marker style string, `~.path.Path` or
`~.markers.MarkerStyle`
markeredgecolor or mec: :mpltype:`color`
markeredgewidth or mew: float
markerfacecolor or mfc: :mpltype:`color`
markerfacecoloralt or mfcalt: :mpltype:`color`
markersize or ms: float
markevery: None or int or (int, int) or slice or list[int] or
float or (float, float) or list[bool]
mouseover: bool
path_effects: list of `~.AbstractPathEffect`
picker: float or callable[[Artist, Event], tuple[bool, dict]]
pickradius: float
rasterized: bool
sketch_params: (scale: float, length: float, randomness: float)
snap: bool or None
solid_capstyle: `CapStyle` or {'butt', 'projecting', 'round'}
solid_joinstyle: `JoinStyle` or {'miter', 'round', 'bevel'}
transform: unknown
url: str
visible: bool
xdata: 1D array
ydata: 1D array
zorder: float

```

See Also

scatter : XY scatter plot with markers of varying size and/or color (sometimes also called bubble chart).

Notes

.. note::

This is the :ref:`pyplot wrapper <pyplot\_interface>` for
`.axes.Axes.plot`.

****Format Strings****

A format string consists of a part for color, marker and line::

```
fmt = '[marker][line][color]'
```

Each of them is optional. If not provided, the value from the style cycle is used. Exception: If ``line`` is given, but no ``marker``, the data will be a line without markers.

Other combinations such as ``[color][marker][line]`` are also supported, but note that their parsing may be ambiguous.

****Markers****

character	description
`.`	point marker
`,`	pixel marker
`o`	circle marker
`v`	triangle_down marker
`^`	triangle_up marker
`<`	triangle_left marker
`>`	triangle_right marker
`1`	tri_down marker
`2`	tri_up marker
`3`	tri_left marker
`4`	tri_right marker
`8`	octagon marker
`s`	square marker
`p`	pentagon marker
`P`	plus (filled) marker
`*`	star marker
`h`	hexagon1 marker
`H`	hexagon2 marker
`+`	plus marker
`x`	x marker
`X`	x (filled) marker
`D`	diamond marker
`d`	thin_diamond marker
` `	vline marker
`_`	hline marker

****Line Styles****

character	description
``_``	solid line style
``_``	dashed line style
``_``	dash-dot line style
``_``	dotted line style

Example format strings::

```
'b'      # blue markers with default shape
'or'     # red circles
'-g'     # green solid line
'--'     # dashed line with default color
'^k:'    # black triangle_up markers connected by a dotted line
```

****Colors****

The supported color abbreviations are the single letter codes

character	color
``b``	blue
``g``	green
``r``	red
``c``	cyan
``m``	magenta
``y``	yellow
``k``	black
``w``	white

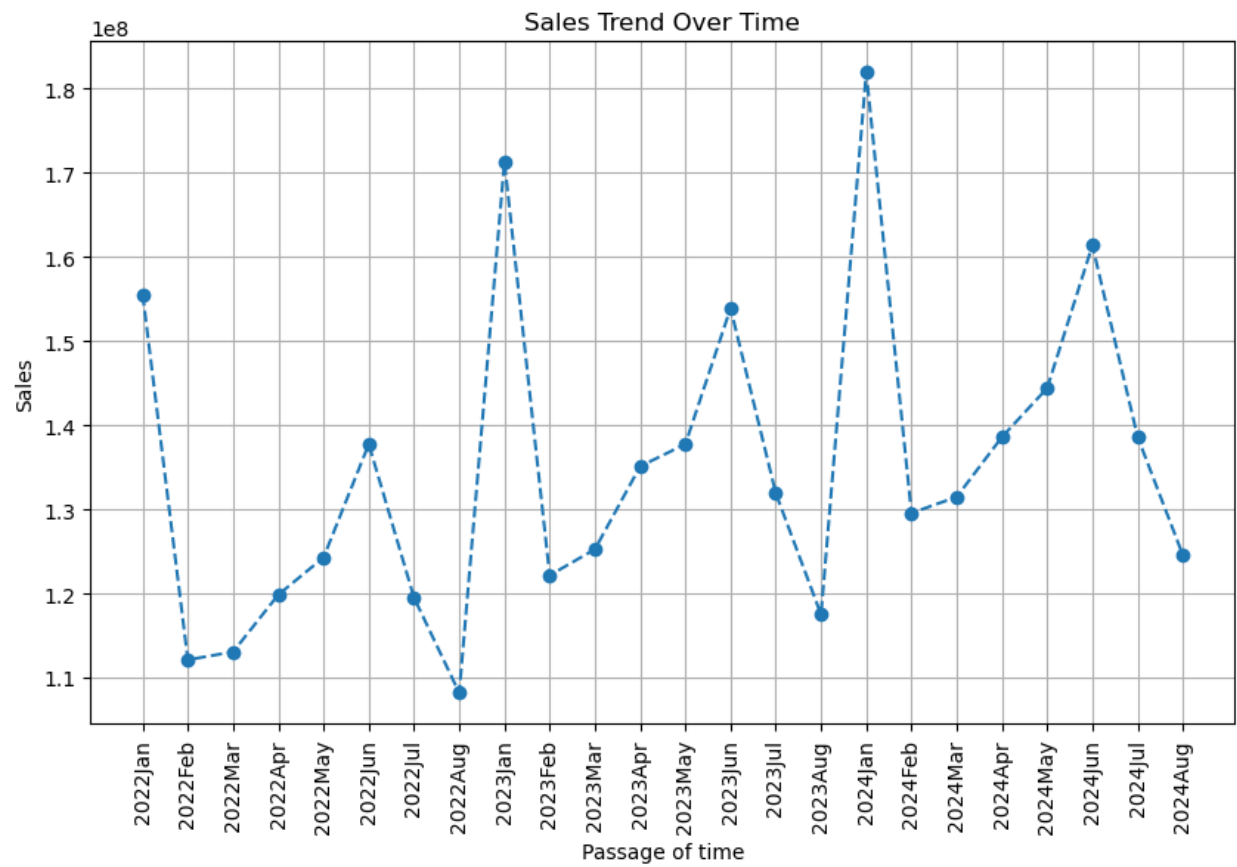
and the ``CN`` colors that index into the default property cycle.

If the color is the only part of the format string, you can additionally use any `matplotlib.colors` spec, e.g. full names (``'green'``) or hex strings (``'#008000'``).

```
File:      c:\users\chukw\anaconda3\lib\site-packages\matplotlib\
pyplot.py
Type:      function
```

```
plt.figure(figsize=(10,6))
plt.plot(monthdf.index, monthdf["Total"], marker="o", linestyle="--")
plt.xticks(rotation="vertical")
plt.title("Sales Trend Over Time")
plt.xlabel("Passage of time")
plt.ylabel("Sales")
```

```
plt.grid(True)
plt.show()
```



chocolate sales constantly peak at january and february of every year due to, chocolate used for valentines day... with record high being in 2024

Q4 Sales Efficiency by sales person

```
choco["Average Revenue per box"]= choco["Amount"]/choco["Boxes Shipped"]
```

choco

	Sales Person	Country	Product	Date
Amount \ 0	Jehu Rudeforth	UK	Mint Chip Choco	2022-01-04
5320.00				
1	Van Tuxwell	India	85% Dark Bars	2022-08-01
7896.00				
2	Gigi Bohling	India	Peanut Butter Cubes	2022-07-07
4501.00				

3	Jan Morforth	Australia	Peanut Butter Cubes	2022-04-27
12726.00				
4	Jehu Rudeforth	UK	Peanut Butter Cubes	2022-02-24
13685.00				
...	...	...	...	...
...				
3277	Karlen McCaffrey	Australia	Spicy Special Slims	2024-05-17
5303.58				
3278	Jehu Rudeforth	USA	White Choc	2024-06-07
7339.32				
3279	Ches Bonnell	Canada	Organic Choco Syrup	2024-07-26
616.09				
3280	Dotty Strutley	India	Eclairs	2024-07-28
2504.62				
3281	Karlen McCaffrey	India	70% Dark Bites	2024-05-23
5915.87				

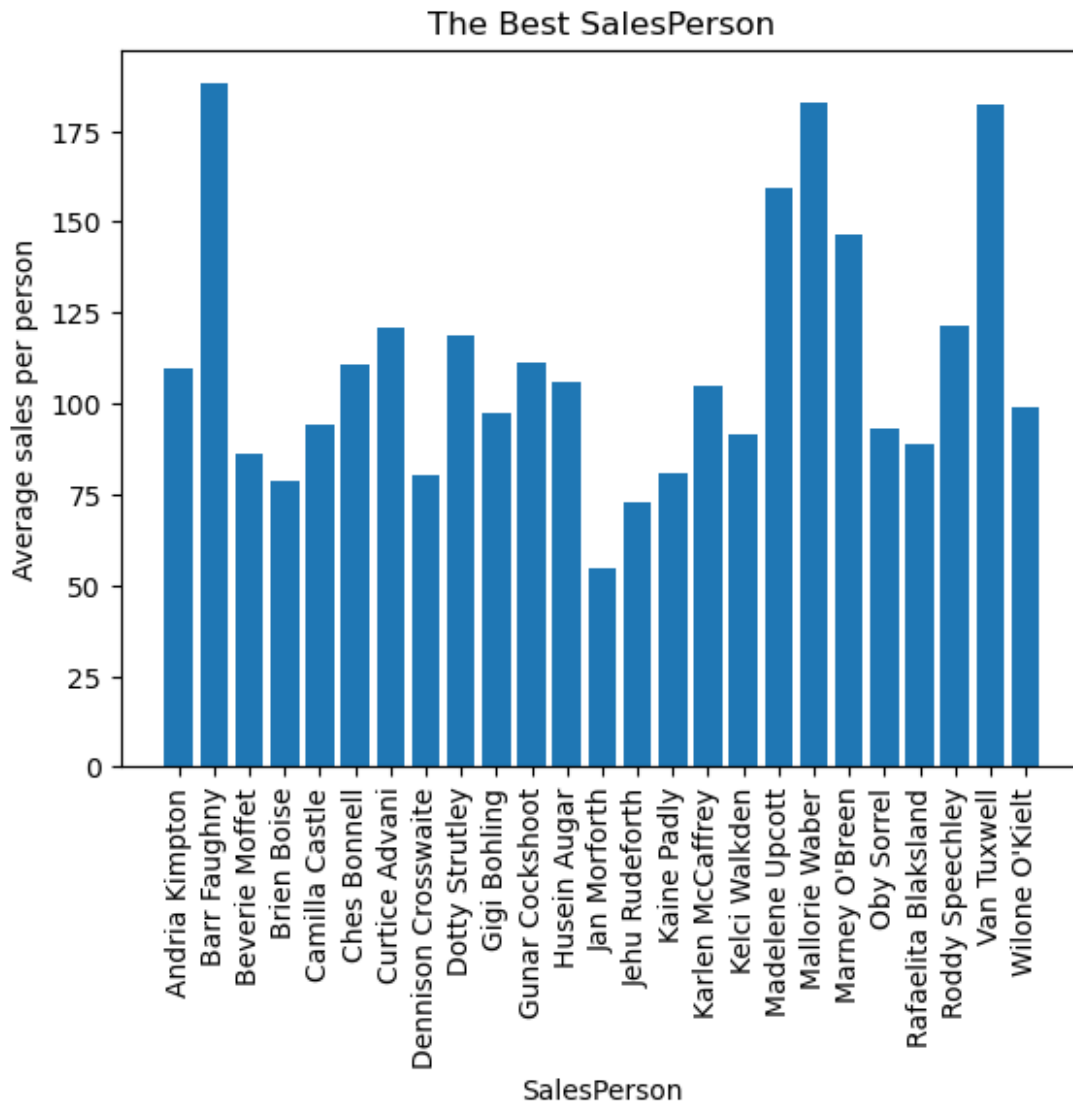
	Boxes Shipped	Day	Month	year	Total	Average Revenue per box
0	180	4	1	2022	957600.00	29.555556
1	94	1	8	2022	742224.00	84.000000
2	91	7	7	2022	409591.00	49.461538
3	342	27	4	2022	4352292.00	37.210526
4	184	24	2	2022	2518040.00	74.375000
...	...	...	...	...	...	...
...						
3277	354	17	5	2024	1877467.32	14.981864
3278	121	7	6	2024	888057.72	60.655537
3279	238	26	7	2024	146629.42	2.588613
3280	397	28	7	2024	994334.14	6.308866
3281	355	23	5	2024	2100133.85	16.664423

[3282 rows x 11 columns]

Average revenue per SalesPerson

```
revenue_df = choco.groupby("Sales Person")[["Average Revenue per box"]].mean()
```

```
plt.bar(revenuedf.index, revenuedf["Average Revenue per box"])
plt.xticks(rotation="vertical")
plt.title("The Best SalesPerson")
plt.xlabel("SalesPerson")
plt.ylabel("Average sales per person")
plt.show()
```



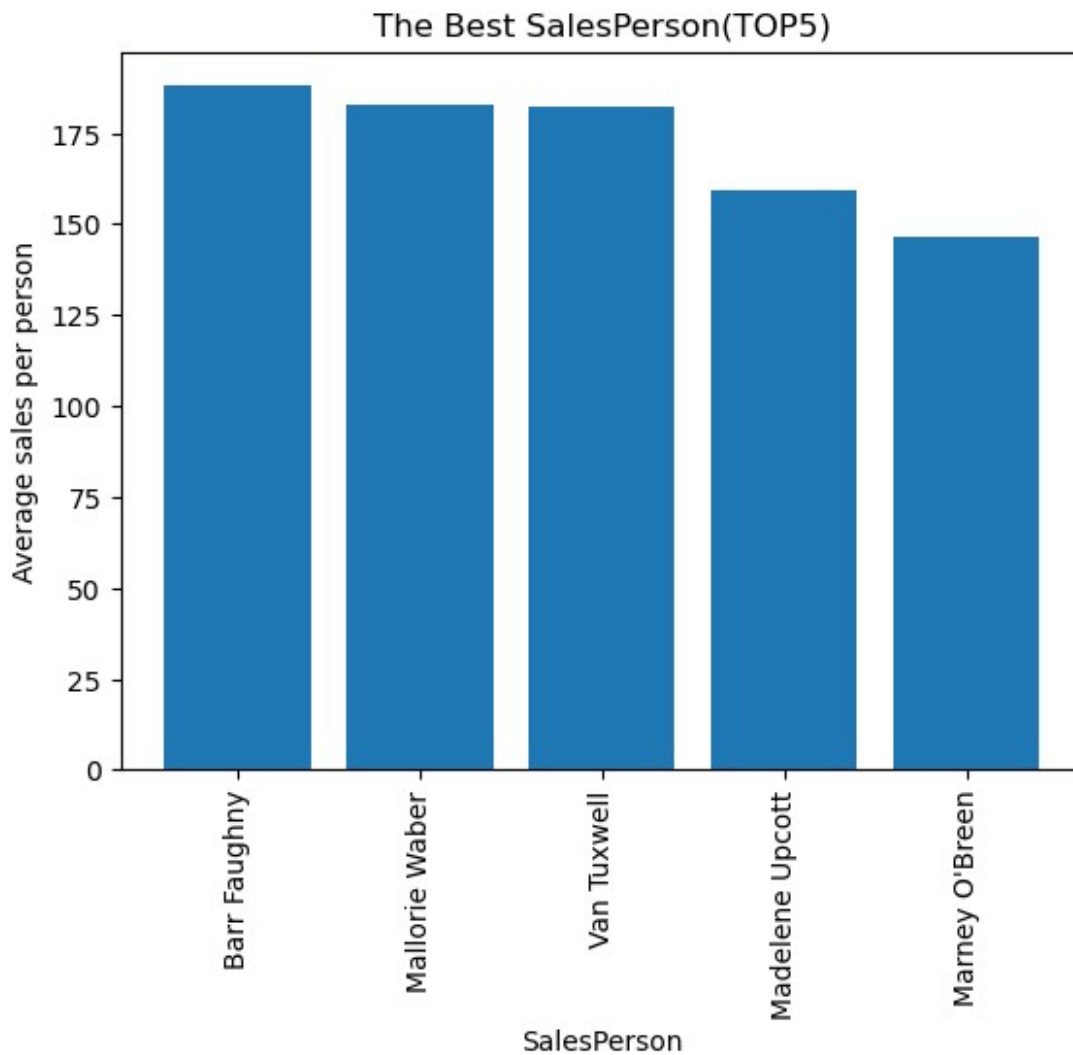
The Most Efficient Sales Person

```
sortedrev = revenuedf.sort_values(by="Average Revenue per box",
ascending = False)

top5=sortedrev.head(5)
top5
```

Average Revenue per box	
Sales Person	
Barr Faughny	187.887583
Mallorie Waber	182.741836
Van Tuxwell	182.350649
Madelene Upcott	159.104593
Marney O'Brien	146.621525

```
plt.bar(top5.index,top5["Average Revenue per box"])
plt.xticks(rotation="vertical")
plt.title("The Best SalesPerson(TOP5)")
plt.xlabel("SalesPerson")
plt.ylabel("Average sales per person")
plt.show()
```



There from the graph above the best salesperson: BARRY FRAUGHNEY